

# Estructures de dades arborescents

Departament de Ciències de la Computació

Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA  
BARCELONATECH

Escola Politècnica Superior d'Enginyeria  
de Vilanova i la Geltrú

# Índex

- 1 Arbres
- 2 Arbres  $n$ -aris
- 3 Arbres binaris
- 4 Ús d'arbres binaris
  - Exemples
  - Recorreguts d'arbres
- 5 Exercicis

# Índex

- 1 Arbres
- 2 Arbres  $n$ -aris
- 3 Arbres binaris
- 4 Ús d'arbres binaris
- 5 Exercicis

# Arbres

## Definició

**Arbre:** Estructura de dades en què els seus elements estan distribuïts de forma jeràrquica. Un element pot tenir més d'un element successor  
⇒ estructura **no** lineal.

# Arbres

## Definició

**Arbre:** Estructura de dades en què els seus elements estan distribuïts de forma jeràrquica. Un element pot tenir més d'un element successor  
⇒ estructura **no** lineal.

- **Node:** element d'un arbre

# Arbres

## Definició

**Arbre:** Estructura de dades en què els seus elements estan distribuïts de forma jeràrquica. Un element pot tenir més d'un element successor  
⇒ estructura **no** lineal.

- **Node:** element d'un arbre
- **Arrel:** únic node que no té pare i punt de partida de tot l'arbre
  - Element de tot arbre no buit

# Arbres

## Definició

**Arbre:** Estructura de dades en què els seus elements estan distribuïts de forma jeràrquica. Un element pot tenir més d'un element successor  
⇒ estructura **no** lineal.

- **Node:** element d'un arbre
- **Arrel:** únic node que no té pare i punt de partida de tot l'arbre
  - Element de tot arbre no buit
  - Node consultable

# Arbres

## Definició

**Arbre:** Estructura de dades en què els seus elements estan distribuïts de forma jeràrquica. Un element pot tenir més d'un element successor  
⇒ estructura **no** lineal.

- **Node:** element d'un arbre
- **Arrel:** únic node que no té pare i punt de partida de tot l'arbre
  - Element de tot arbre no buit
  - Node consultable
  - Pot estar connectat amb zero o més arbres anomenats **fills**



# Arbres

## Definició

**Arbre:** Estructura de dades en què els seus elements estan distribuïts de forma jeràrquica. Un element pot tenir més d'un element successor  
⇒ estructura **no** lineal.

- **Node:** element d'un arbre
- **Arrel:** únic node que no té pare i punt de partida de tot l'arbre
  - Element de tot arbre no buit
  - Node consultable
  - Pot estar connectat amb zero o més arbres anomenats **fills**
- **Fulla** o node terminal: node amb fills buits

# Arbres

- **Camí:** successió de nodes que van des de l'arrel a una fulla

# Arbres

- **Camí:** successió de nodes que van des de l'arrel a una fulla
- **Altura:** longitud del camí més llarg

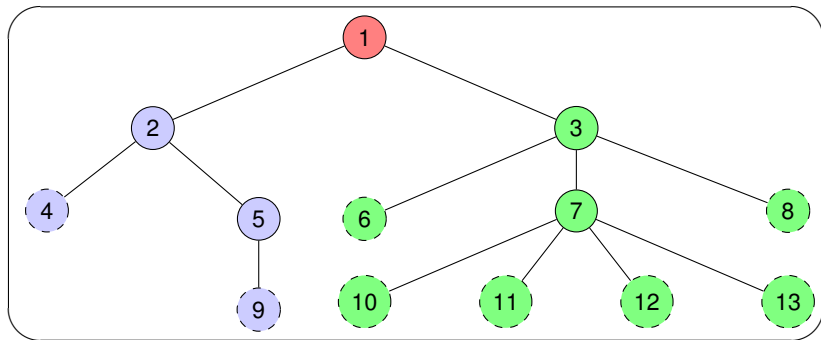
# Arbres

- **Camí:** successió de nodes que van des de l'arrel a una fulla
- **Altura:** longitud del camí més llarg
- S'assembla a un arbre normal (arrel, fulles, etc) però invertit

# Arbres

- **Camí:** successió de nodes que van des de l'arrel a una fulla
- **Altura:** longitud del camí més llarg
- S'assembla a un arbre normal (arrel, fulles, etc) però invertit
- **Accés:** es pot consultar l'arrel i després accedir a cada arbre fill individualment

# Exemple d'arbre general



- arrel
- subarbre esquerre, fill esquerre, primer fill
- subarbre dret, fill dret, segon fill
- fulles

# Índex

1 Arbres

2 Arbres  $n$ -aris

3 Arbres binaris

4 Ús d'arbres binaris

5 Exercicis

## Arbres $n$ -aris

- Tipus d'arbre on tots els subarbres no buits tenen exactament el mateix nombre de fills, siguin aquests fills arbres buits o no.

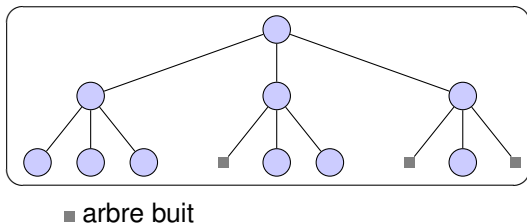


## Arbres $n$ -aris

- Tipus d'arbre on tots els subarbres no buits tenen exactament el mateix nombre de fills, siguin aquests fills arbres buits o no.
- $n$  és un paràmetre del constructor de la classe, indica el nombre de fills.

# Arbres $n$ -aris

- Tipus d'arbre on tots els subarbres no buits tenen exactament el mateix nombre de fills, siguin aquests fills arbres buits o no.
- $n$  és un paràmetre del constructor de la classe, indica el nombre de fills.
- Exemple:*  $n = 3$



# Índex

1 Arbres

2 Arbres  $n$ -aris

3 Arbres binaris

4 Ús d'arbres binaris

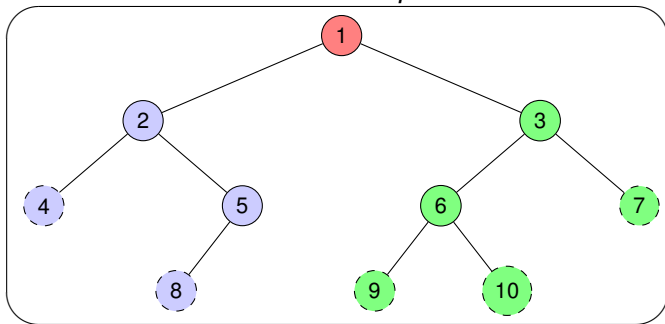
5 Exercicis

# Arbres binaris

- Arbres  $n$ -aris on  $n = 2$ .

# Arbres binaris

- Arbres  $n$ -aris on  $n = 2$ . *Exemple:*



- arrel
- subarbre esquerre, la seva arrel és 2
- subarbre dret, la seva arrel és 3
- fulles (els arbres buits no s'han representat)

# Especificació de la classe genèrica `arbreBin`

```
template <typename T> class arbreBin{  
    // Tipus de mòdul: dades  
    // Descripció del tipus: Arbre genèric que o bé és buit  
    // o bé tot subarbre seu té exactament dos fills.  
  
private:  
  
public:  
    // Constructors  
    // Modificadors  
    // Consultors  
    // Lectura i escriptura  
};
```

# Constructors de arbreBin

```
arbreBin();  
/* Pre: cert */  
/* Post: El resultat és un arbre sense cap element */  
  
arbreBin(const arbreBin<T> &a);  
/* Pre: cert */  
/* Post: El resultat és un arbre còpia de l'arbre rebut */  
  
arbreBin(const T &x, const arbreBin<T> &ae, const arbreBin<T> &ad);  
/* Pre: cert */  
/* Post: El resultat és un arbre amb x com a arrel, ae com a fill  
esquerre i ad com a fill dret */
```

# Destructor de arbreBin

```
// Destructor: Esborra automàticament els objectes locals  
// en sortir d'un àmbit de visibilitat  
~arbreBin();
```



# Modificadors de arbreBin

```
void a_buit();  
/* Pre: cert */  
/* Post: El paràmetre implícit no té cap element */  
  
arbreBin<T>& operator=(const arbreBin<T> &a);  
/* Pre: cert */  
/* Post: Operació d'assignació d'arbres */
```

# Consultors de arbreBin

```
const T& arrel() const;
/* Pre: El paràmetre implícit no és buit */
/* Post: El resultat és l'arrel del paràmetre implícit */

arbreBin<T> fe() const;
/* Pre: El paràmetre implícit no és buit */
/* Post: El resultat és el fill esquerre del paràmetre implícit */

arbreBin<T> fd() const;
/* Pre: El paràmetre implícit no és buit */
/* Post: El resultat és el fill dret del paràmetre implícit */

bool es_buit() const;
/* Pre: cert */
/* Post: El resultat indica si el paràmetre implícit és buit o no */
```

# Lectura i escriptura arbreBin

```
// operador << d'escriptura

template <class U>
friend std::ostream& operator<<(std::ostream&, const arbreBin<U> &a);
/* Pre: cert */
/* Post: s'han escrit al canal estandard de sortida els elements d'a*/

// operador >> de lectura

template <class U>
friend std::istream& operator>>(std::istream&, arbreBin<U> &a);
/* Pre: a és buit; el canal estàndard d'entrada conté la mida de
   l'arbre i els nodes en postordre d'un arbre binari A; per cada
   node s'indica el seu valor i el nombre de fills */
/* Post: a = A */
```

# Índex

- 1 Arbres
- 2 Arbres  $n$ -aris
- 3 Arbres binaris
- 4 Ús d'arbres binaris**
  - Exemples
  - Recorreguts d'arbres
- 5 Exercicis

# Ús de la classe `arbreBin`

- Instanciació: `arbreBin<T> nom_arbre;`

# Ús de la classe `arbreBin`

- Instanciació: `arbreBin<T> nom_arbre;`
- *Exemples:*

# Ús de la classe `arbreBin`

- Instanciació: `arbreBin<T> nom_arbre;`

- *Exemples:*

```
arbreBin<int> a;
```

```
arbreBin<Punt> ap;
```

# Índex

1 Arbres

2 Arbres  $n$ -aris

3 Arbres binaris

4 Ús d'arbres binaris

- Exemples
- Recorreguts d'arbres

5 Exercicis



# Exemple 1: mida d'un arbre binari (versió iterativa)

```

int it_mida(const arbreBin<int>& a)
/* Pre: a = A */
/* Post: El resultat és el nombre de nodes de l'arbre A */
{
    stack<arbreBin<int>> p;
    p.push(a);

    int nnodes = 0;
    while (not p.empty()) {
        arbreBin<int> a1 = p.top();
        p.pop();
        if (not a1.es_buit()) {
            ++nnodes;
            if (not a1.fe().es_buit()) p.push(a1.fe());
            if (not a1.fd().es_buit()) p.push(a1.fd());
        }
    }
    return nnodes;
}

```

# Exemple 1: mida d'un arbre binari (versió recursiva)

```

int mida(const arbreBin<int>& a)
/* Pre: a = A */
/* Post: El resultat és el nombre de nodes de l'arbre A */
{
    int nnodes;

    if (a.es_buit()) nnodes = 0;
    else{
        arbreBin<int> a1 = a.fe();
        arbreBin<int> a2 = a.fd();
        int y = mida(a1);
        int z = mida(a2);
        nnodes = y + z + 1;
    }
    return nnodes;
}

```

## Exemple 2: cerca en un arbreBin<int>

```

bool it_cerca(const arbreBin<int>& a, int x)
/* Pre: a = A */
/* Post: El resultat indica si x és a l'arbre A o no */
{
    stack<arbreBin<int>> p;

    p.push(a);
    bool trobat = false;

    while (not p.empty() and not trobat) {
        arbreBin<int> a1 = p.top();
        p.pop();
        if (not a1.es_buit()) {
            if (a1.arrel() == x) trobat = true;
            else {
                if (not a1.fe().es_buit()) p.push(a1.fe());
                if (not a1.fd().es_buit()) p.push(a1.fd());
            }
        }
    }
    return trobat;
}

```

## Exercici: nombre de fulles d'un arbre binari

```
int it_fulles(const arbreBin<int>& a)
/* Pre: a = A */
/* Post: El resultat és el nombre de fulles de l'arbre A */
{
    stack<arbreBin<int>> p;
    p.push(a);
    ...

    while (not p.empty()) {
        ...
    }
}
return nfulles;
}
```

# Índex

1 Arbres

2 Arbres  $n$ -aris

3 Arbres binaris

4 Ús d'arbres binaris

- Exemples

- Recorreguts d'arbres

5 Exercicis

# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.

# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.
- Un node es visita per modificar-lo o perquè forma part del càlcul d'una funció.

# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.
- Un node es visita per modificar-lo o perquè forma part del càlcul d'una funció.
- Els mètodes es diferencien en l'**ordre** de visita dels nodes.



# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.
- Un node es visita per modificar-lo o perquè forma part del càlcul d'una funció.
- Els mètodes es diferencien en l'**ordre** de visita dels nodes.
- Recorreguts bàsics:

# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.
- Un node es visita per modificar-lo o perquè forma part del càlcul d'una funció.
- Els mètodes es diferencien en l'**ordre** de visita dels nodes.
- Recorreguts bàsics:
  - En profunditat:

# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.
- Un node es visita per modificar-lo o perquè forma part del càlcul d'una funció.
- Els mètodes es diferencien en l'**ordre** de visita dels nodes.
- Recorreguts bàsics:
  - En profunditat:
    - preordre

# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.
- Un node es visita per modificar-lo o perquè forma part del càlcul d'una funció.
- Els mètodes es diferencien en l'**ordre** de visita dels nodes.
- Recorreguts bàsics:
  - En profunditat:
    - preordre
    - inordre

# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.
- Un node es visita per modificar-lo o perquè forma part del càlcul d'una funció.
- Els mètodes es diferencien en l'**ordre** de visita dels nodes.
- Recorreguts bàsics:
  - En profunditat:
    - preordre
    - inordre
    - postordre

# Recorreguts d'arbres

- Diversos mètodes per visitar els nodes d'un arbre per fer recorreguts o cerques.
- Un node es visita per modificar-lo o perquè forma part del càlcul d'una funció.
- Els mètodes es diferencien en l'**ordre** de visita dels nodes.
- Recorreguts bàsics:
  - En profunditat:
    - preordre
    - inordre
    - postordre
  - En amplitud o per nivells

# Recorreguts d'arbres: preordre

# Recorreguts d'arbres: preordre

1 Visitar l'arrel.



# Recorreguts d'arbres: preordre

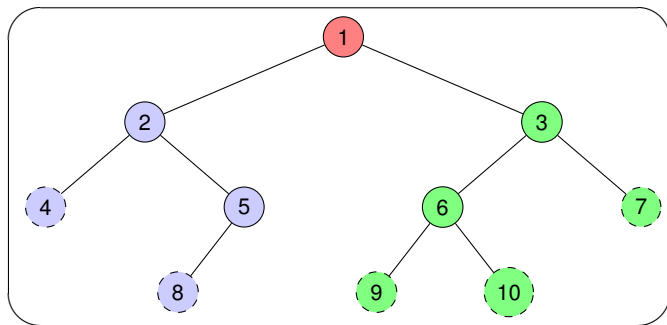
- 1 Visitar l'arrel.
- 2 Recórrer en preordre el subarbre esquerre.

# Recorreguts d'arbres: preordre

- 1 Visitar l'arrel.
- 2 Recórrer en preordre el subarbre esquerre.
- 3 Recórrer en preordre el subarbre dret.

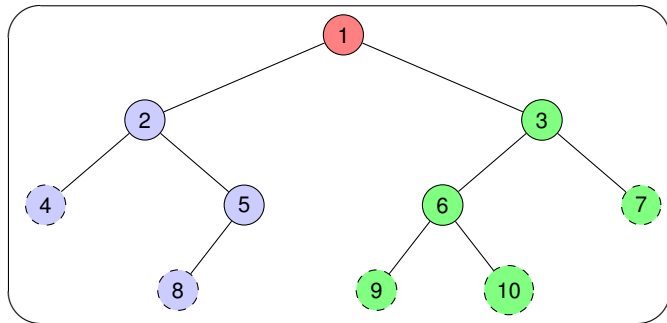
# Recorreguts d'arbres: preordre

- 1 Visitar l'arrel.
- 2 Recórrer en preordre el subarbre esquerre.
- 3 Recórrer en preordre el subarbre dret. *Exemple:*



# Recorreguts d'arbres: preordre

- 1 Visitar l'arrel.
- 2 Recórrer en preordre el subarbre esquerre.
- 3 Recórrer en preordre el subarbre dret. *Exemple:*



Recorregut en preordre: 1, 2, 4, 5, 8, 3, 6, 9, 10, 7

# Recorreguts d'arbres: inordre

# Recorreguts d'arbres: inordre

- 1 Recórrer en inordre el subarbre esquerre.

# Recorreguts d'arbres: inordre

- 1 Recórrer en inordre el subarbre esquerre.
- 2 Visitar l'arrel.

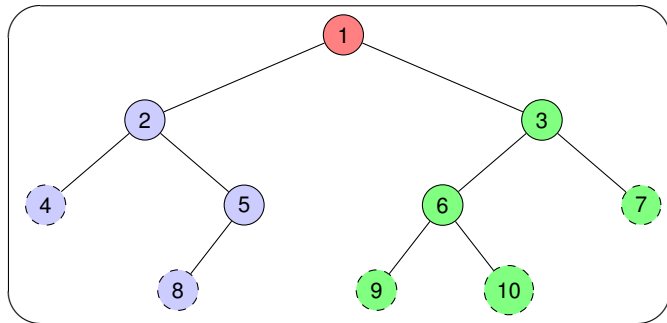
# Recorreguts d'arbres: inordre

- 1 Recórrer en inordre el subarbre esquerre.
- 2 Visitar l'arrel.
- 3 Recórrer en inordre el subarbre dret.



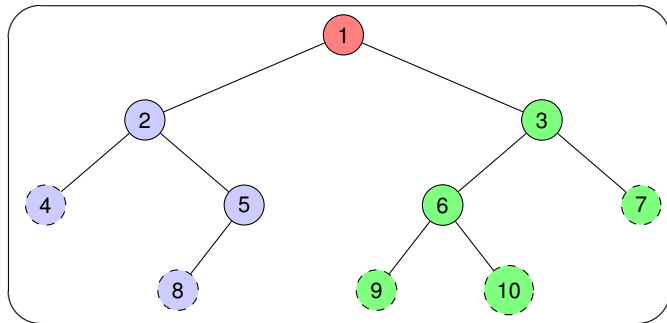
# Recorreguts d'arbres: inordre

- 1 Recórrer en inordre el subarbre esquerre.
- 2 Visitar l'arrel.
- 3 Recórrer en inordre el subarbre dret. *Exemple:*



# Recorreguts d'arbres: inordre

- 1 Recórrer en inordre el subarbre esquerre.
- 2 Visitar l'arrel.
- 3 Recórrer en inordre el subarbre dret. *Exemple:*



Recorregut en inordre: 4, 2, 8, 5, 1, 9, 6, 10, 3, 7

# Recorreguts d'arbres: postordre

# Recorreguts d'arbres: postordre

- 1 Recórrer en postordre el subarbre esquerre.

## Recorreguts d'arbres: postordre

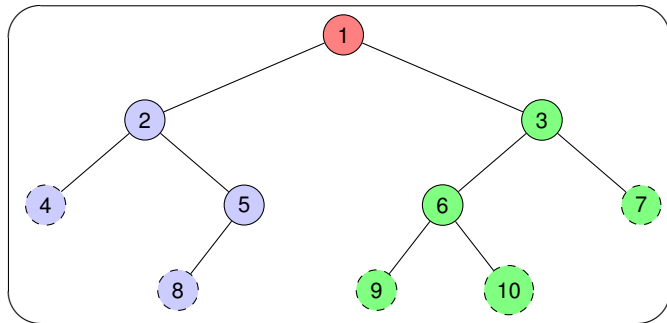
- 1 Recórrer en postordre el subarbre esquerre.
- 2 Recórrer en postordre el subarbre dret.

# Recorreguts d'arbres: postordre

- 1 Recórrer en postordre el subarbre esquerre.
- 2 Recórrer en postordre el subarbre dret.
- 3 Visitar l'arrel.

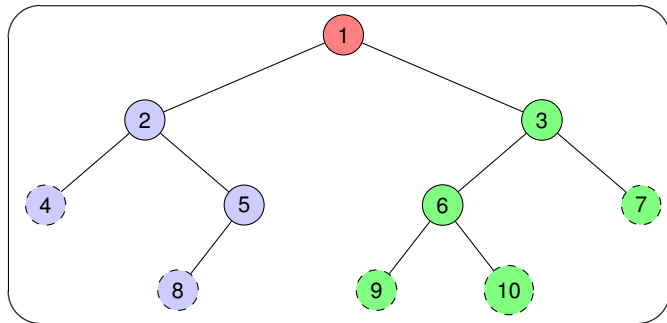
# Recorreguts d'arbres: postordre

- 1 Recórrer en postordre el subarbre esquerre.
- 2 Recórrer en postordre el subarbre dret.
- 3 Visitar l'arrel. *Exemple:*



# Recorreguts d'arbres: postordre

- 1 Recórrer en postordre el subarbre esquerre.
- 2 Recórrer en postordre el subarbre dret.
- 3 Visitar l'arrel. *Exemple:*



Recorregut en postordre: 4, 8, 5, 2, 9, 10, 6, 7, 3, 1



# Exemple: recorregut en preordre d'un arbre

```

list<int> it_preordre(const arbreBin<int>& a)
/* Pre: a = A */
/* Post: El resultat conté els nodes d'A en preordre */
{
    stack<arbreBin<int>> p;

    p.push(a);
    list<int> l;

    while (not p.empty()) {
        arbreBin<int> a1 = p.top();
        p.pop();
        if (not a1.es_buit()) {
            l.insert(l.end(), a1.arrel());
            if (not a1.fd().es_buit()) p.push(a1.fd());
            if (not a1.fe().es_buit()) p.push(a1.fe());
        }
    }
    return l;
}

```

# Recorregut en amplada d'un arbre

- Recorregut en amplada o per nivells (*breadth first traversal*).

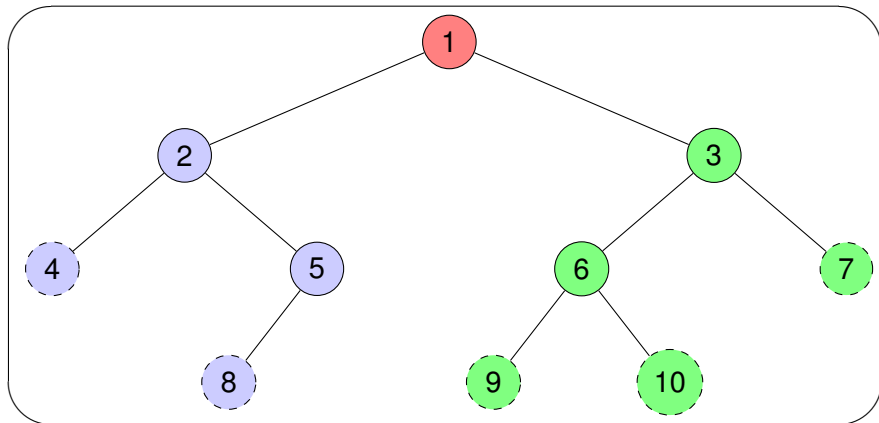
# Recorregut en amplada d'un arbre

- Recorregut en amplada o per nivells (*breadth first traversal*).
- Nivell d'un node: distància a l'arrel (nombre d'enllaços).

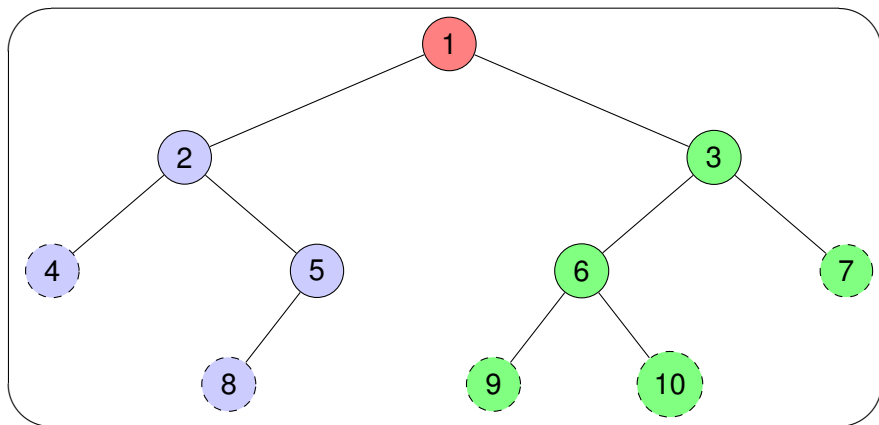
# Recorregut en amplada d'un arbre

- Recorregut en amplada o per nivells (*breadth first traversal*).
- Nivell d'un node: distància a l'arrel (nombre d'enllaços).
- Passos:
  - 1 Arrel de l'arbre (node de nivell 0)
  - 2 Tots els nodes que estan al nivell 1, d'esquerra a dreta.
  - 3 Tots els nodes que estan a nivell 2, també d'esquerra a dreta.
  - 4 Etc.

# Exemple: recorregut en amplada d'un arbre



# Exemple: recorregut en amplada d'un arbre



Recorregut en amplada: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

# Exemple: recorregut en amplada d'un arbre

```

void amplada(const arbreBin<int> &a, list<int> &l)
/* Pre: a = A, l és buida */
/* Post: l conté els nodes d'A en ordre creixent respecte al nivell
en què es troben, i els de cada nivell en ordre d'esquerra a dreta */
{
    if (not a.es_buit()) {
        queue <arbreBin<int>> c;
        c.push(a);

        while (not c.empty()) {
            l.insert(l.end(), c.front().arrel());
            arbreBin<int> a1 = c.front().fe();
            arbreBin<int> a2 = c.front().fd();
            if (not a1.es_buit()) c.push(a1);
            if (not a2.es_buit()) c.push(a2);
            c.pop();
        }
    }
}

```

# Índex

- 1 Arbres
- 2 Arbres  $n$ -aris
- 3 Arbres binaris
- 4 Ús d'arbres binaris
- 5 Exercicis



# Exercicis

- Resoleu els exercicis proposats en aquesta sessió.
- Implementeu els recorreguts en inordre i en postordre de forma iterativa.
- Resoleu exercicis sobre arbres de la col·lecció de problemes.